

Speculative activity: Theseus works ahead, in a sandbox, and offers the result

Epic theseus-upf0, a child of the north star theseus-3iwi. Written 2026-10-09 by Tabitha/Claude, read-only against origin/main d348ac67 (every code citation is unchanged since 69cb5375). A design only: nothing here is built yet. The owner took every default on 2026-10-09 (section 8); the rows are filed (section 7) and start after the benchmark rerun (theseus-lc7a).

| The answer first

The idea. It came from a collaborator: *"speculation, where it tries to guess what you're going to ask and tries a few different things so its got them cached"*. The owner shaped it: *"theseus realizes an end to end that the user may be heading towards, and safely, in a suitable style of sandboxing, blazes forward, then offering to the user if it would be useful"*.

The pitch. While you read Theseus's last answer or step away, it guesses what you'll ask next from the red test, the plan's next step, the task board and your habits, and quietly does that work in a throwaway copy with no network, no secrets and no way to touch your files. When you come back, or when you ask for exactly that, the answer, the diff and the passing tests are already there: one key to use it, one to throw it away. It runs only when your machine is idle, spends from its own small daily budget, and every guess is scored, so it gets better at guessing and stops guessing where it is wrong.

What is decided. \$10 a day for all speculation; rungs S0 to S2 on in private places once their shadow phase passes; nothing applied to the owner's tree without **Use**; a speculation lives 8 hours at most; no network for its builds. Out of scope: speculating for anyone but the owner, speculation in shared places, and Theseus improving its own code (that is [the self-improvement plan](#), whose governance this borrows).

1. What already exists to build on

Piece	What exists (origin/main)	What speculation takes from it	Gap
L1 sandbox	A job in its own user, pid, mount, network, uts, ipc and cgroup namespaces; tmpfs root; read-only system binds; overlays over the workspace whose writes are scratch; no capabilities, <code>no_new_privs</code> , a seccomp deny list (<code>crates/theseus-sandbox/src/lib.rs:1-22</code>). Theseus's floor, store, spool, token and socket hidden (<code>Spec::hidden</code>). Scratch is a 1024 MB tmpfs, discarded at exit (<code>init.rs:526-570</code>). No network unless <code>[sandbox] egress</code> or the call names hosts (<code>crates/theseus-core/src/egress.rs:1-22</code>).	S1's scouting runs as-is; S2's builds and tests run here.	L1 covers <code>proc.run</code> only, while the daemon's own <code>fs.*</code> tools write the real tree; no read-write bind, so a build tree cannot persist between one speculation's jobs.
Tasks	<code>task.create {brief, budget_usd?}</code> opens a child session in one kernel frame (<code>crates/theseus-kernel/src/tasks.rs:90</code>); it inherits authority, persona, postures and model; its budget is carved from the parent's; its arrangement starts it from quoted pieces of the parent session; its last message is its report (<code>crates/theseus-core/src/task.rs:1-40</code>).	The shape of a speculation session: a child with a carved budget, an arranged brief, a report.	A task reports into its parent and inherits its whole authority; a speculation must do neither until accepted.
Authority ceilings	<code>Authority { principal, delegated_by, ceilings }</code> , "inherited never widened", intersected on fork (<code>crates/theseus-kernel/src/types.rs:175-184</code>).	The narrowing mechanism: a speculation carries a <code>spec ceiling</code> .	No <code>spec ceiling</code> is read by the gate today.
Places	Private and shared places; shared places get no <code>proc.run</code> , no <code>aws.*</code> , no owner material (<code>crates/theseus-core/src/places.rs:1-35</code>); a place's ceiling narrows tools, posture, spend and profile (<code>crates/theseus-core/src/ceiling.rs:1-40</code>).	Speculation only in private places.	Withholding tools changes the prompt prefix (section 4.1: the catalog must stay identical so the cache holds).
The gate	Postures <code>Open</code> , <code>Notify</code> , <code>Approve</code> (<code>crates/theseus-core/src/policy.rs:47-80</code>); a call's posture is the strictest of config, tightening, floor and the outside-text hold.	What may run without asking.	A speculation must never ask: an <code>Approve</code> call is refused and its reason lands in the offer.
Secret broker	Grants by name to a program at run time (<code>crates/theseus-core/src/broker.rs</code>).	Nothing: a speculation gets no grant.	A <code>spec ceiling</code> that empties the grant set.
Outside text	A job that reached a host beyond the operator's list returns untrusted <code>external</code> text and holds the session (<code>egress.rs:14-22</code>).	Speculation reads no outside text, so it never carries injected instructions into an offer.	none
Wakes	<code>wake.at</code> one-shot wakes, durable across restarts (<code>task.rs:25-31</code>); due wakes ran 0.35 s after serving (north-star plan §1.6).	Time-of-day predictions are scheduled as internal wakes.	none

Piece	What exists (origin/main)	What speculation takes from it	Gap
Background yields	<code>theseus_store::pressure</code> : background passes wait while PSI some <code>avg10</code> is at or over CPU 20% or IO 10% (<code>crates/theseus-store/src/pressure.rs:1-40</code>); <code>idle_this_thread</code> takes <code>SCHED_IDLE</code> ; the CPU pool caps in-process work at one permit a core (<code>crates/theseus-core/src/cpu.rs:1-20</code>).	The idle rule.	It waits at most 10 s and then runs; a speculation must instead <i>not run</i> while busy, and be pre-empted when the owner moves.
Spend	A daily ceiling, \$200 by default, over every model call (<code>crates/theseus-kernel/src/day_ceiling.rs:1-30</code>); budgets reserve before a call.	A speculation account under the day ceiling, like the RSI plan's <code>self-budget</code> .	No speculation account.
Memory	Recall runs in front of the model inside a 250 ms deadline; nightly consolidation writes cited syntheses; <code>classify.v1</code> labels every human message <code>new_ask</code> , <code>follow_up</code> , <code>correction</code> , <code>control</code> , <code>addressed_to_task</code> or <code>social</code> (<code>crates/theseus-judge/packs/classify.v1.toml</code>).	Habit signals; the label that scores a prediction.	No store of predictions and their outcomes.
The work board	One <code>WorkView</code> model for conversations, tasks, steps, jobs, questions and wakes (<code>crates/theseus-protocol/src/work.rs:1-30</code>); the board itself is a later row.	A "Ready ahead" section and a <code>speculation</code> kind.	Not built yet.
Fork and rewind	Planned: <code>session.fork {session, at}</code> by reference, <code>--worktree</code> , and <code>fs.write</code> pre-images (rewind, <code>theseus-23rk</code>).	A speculation is a fork at "now" that the owner never sees unless it lands; pre-images make Use undoable.	Not built.
Keep-warm and cache (theseus-ezeg)	Found: the <code>fable</code> , <code>opus</code> and <code>haiku</code> profiles cached for 5 minutes; a cold rewrite of the owner's ~626K-token DM prompt is about \$7.85, a warm read about \$0.16. Approved: an hourly keep-warm read for up to 24 h and a 1 h TTL.	S0's cache warming <i>is</i> this row. The prediction call rides on a keep-warm read.	none (in the stopping point)
Parallel tool calls (theseus-dlhi, theseus-2wxa)	Independent Run and Write calls of one response run together; a two-layer benchmark (stand-in model for the harness, real model for behaviour).	S3 runs its futures in parallel; the <code>specbench</code> copies the two-layer shape.	none
AWS hands	<code>aws.hands.run</code> on Lambda or Fargate with per-dispatch keys, hour and day lines, runaway mode (<code>crates/theseus-core/src/aws/hands/</code>).	S4.	The hands are not live yet.

Two facts the design turns on.

1. *Isolation is half built.* L1 isolates programs well, but the daemon's own `fs.*` writes the real tree, and a build tree cannot live in a 1 GB tmpfs. So S2 needs a **speculation workspace** (section 4.2).

2. *The cache decides the price.* A speculation that sends the owner's session prefix byte for byte reads the cache (about \$0.16 for a 626K-token DM); one that changes the tool list or the system prompt rewrites it (about \$7.85). Anthropic's cache covers tools, then system, then messages, so a speculation keeps the **same catalog and system prompt** and narrows at the gate, never by withholding tools.

2. The ladder of speculation

Cheapest and safest first. Each rung is its own switch, placed off, shadow or live by its numbers.

Rung	What it does	Cost	Risk	Needs
S0 warm	Keeps the next answer fast without doing the work: the prompt cache warm; recall pre-run for each predicted ask and its hits held for the next turn; the files a prediction names read into the page cache and their index segments touched; the provider connection opened when the owner starts typing.	~\$0 to \$0.02 a prediction	Near none: reads only, nothing shown	The prediction; the cache fix; a typing signal from each surface
S1 read-only scouting	Gathers what the next turn would gather first: runs the tests or the failing one, reads build and job logs, <code>git status</code> and <code>log</code> , greps the symbol in play. Keeps a scout note . No model loop for the deterministic part; at most one call to pick the commands.	\$0 to \$0.10; CPU at <code>SCHED_IDLE</code> in L1	Low: L1, workspace read-only, scratch discarded, no network, no secrets	L1 as built; a scout-note store; the idle rule
S2 sandboxed end to end	Does the likely next task in a throwaway copy: edits files in the speculation workspace, builds and tests in L1, and stops with a ready result : the reply it would give, a diff, its test results, its cost and its base.	\$0.30 to \$1.50, capped	Medium, contained: a copy, no network, no secrets, no messages	The speculation workspace; a <code>spec</code> ceiling; pre-images for Use; the account
S3 several futures, ranked	When no single prediction is strong but the top few together are, or one ask has two plausible approaches: up to three S2s in parallel, ranked by tests, diff size and prediction score.	k × S2 under one \$3 cap	As S2, times k	S2; parallel calls; fork by reference so the futures share one cached prefix
S4 on the AWS hands	Heavy speculation (a test matrix, a long benchmark, a big build) on a hand with no route out but the owner's own bucket.	The hands' own line; off by default	Higher: the copy leaves the machine, to the owner's own account	The hands live; a speculation sub-line; S2

"**End to end**" is the target for S2: not a hint but a finished piece of work as the next turn would have produced it, including the reply text, so **Use** is instant. S0 and S1 exist because most of the value of guessing right is in the first seconds of the next turn, and they are nearly free.

3. Prediction: where "the likely next ask" comes from

3.1 Signals

Signal	Source	Cost	Example
A failing build or test	The last turn's tool results; a job's completion; the gate's red	\$0	"Fix the failing <code>egress_refused</code> test"

Signal	Source	Cost	Example
The task board	A task failed or blocked, a question waiting, a reminder due within the hour, a watch that fired	\$0	"Retry the check after the 401"
Repository state	Uncommitted changes with green tests; a branch ahead of its remote; a review that came in	\$0	"Commit and push it"; "address the review"
The conversation's trajectory	A plan with steps left, "next I'll ..." in the reply, a question the reply asked	one model call	"Do step 3 of the plan"
The owner's habits	Consolidation's syntheses; what usually follows what in <code>classify.v1</code> 's history	\$0 at read time	"Install and restart the daemon"
Time of day	The ledger's history of the owner's first asks by hour and weekday	\$0	"The morning status", gathered at 07:50

The deterministic signals cost nothing and are often the strongest: a red test is followed by "fix it" more often than any model would guess. The model call adds the trajectory.

3.2 The prediction call

At the first idle tick after a reply ends (45 s with no typing and no input), Theseus makes **one off-transcript call**: the session's exact prefix, so it is a cache read and refreshes the 5-minute TTL as a side effect, plus a short fixed suffix asking for up to three likely next asks as structured output (`{ask, why, confidence, kind: answer|code|ops, files[]}`), at most ~300 output tokens. Its result is a `spec.predicted` row, never a node in the session, so the next real turn's prefix is unchanged. The hourly keep-warm reads re-predict only if something changed.

Minimal prompting, in the owner's order of levers: the suffix is a fixed, factual request for a JSON object with a schema; no examples, no persuasion; it is never tuned on the specbench's scenarios.

3.3 Thresholds and the score

Rung	Starts when
S0	any prediction ≥ 0.2
S1	a prediction ≥ 0.3 whose <code>kind</code> needs facts from the machine
S2	a prediction ≥ 0.5 , <code>kind</code> code or ops, and positive expected value: $p \times \text{minutes saved} \times \text{the owner's value of a minute} > \text{the run's cap}$
S3	the top prediction < 0.5 and the top three ≥ 0.7 , or one prediction with two plausible approaches
S4	as S2, plus heavy work, plus S4 on for the place

Scoring (`spec.scored`). When the owner's next input arrives in that session within the prediction's lifetime, each open prediction is scored **hit** (the owner pressed Use, or the input is a `follow_up` or `new_ask` that matches by the index's embedder, ties confirmed by one judge question in the `inbound` call classify already makes), **partial** (same topic, different ask), or **miss**.

Calibration. Each signal source keeps its hit rate over its last 50 scored predictions, and its stated confidence is replaced by its observed rate. A source starts in **shadow** (it predicts and is scored, and nothing above 50 runs on it) until 20 predictions are scored. A rung whose 7-day value (accepted runs' minutes saved) falls under its spend lowers itself one rung and says so in the ledger: placed by numbers, lowered without asking, never raised past the config.

4. Safety

4.1 The envelope: nothing leaves until the owner accepts

A speculation is an execution whose authority carries a `spec` ceiling (intersected, never widened). The gate reads it before anything else:

Effect	Rule in a speculation
Writes to the owner's tree	Refused. <code>fs.*</code> writes only inside the speculation workspace; any path outside it is refused outright.
Programs	L1 only , with the speculation workspace read-write and nothing else; <code>sandbox: false</code> and L0 are refused.
Network	None , the operator's <code>[sandbox] egress</code> included; <code>web.*</code> and <code>http.fetch</code> are refused.
Messages	None. <code>channel.post</code> , reads into another place, <code>task.create</code> , <code>wake.*</code> , <code>extend.*</code> and every MCP tool that is not read-only are refused.
Secrets	None. The broker grants nothing; approve-list paths stay hidden.
Installs	None outside the workspace (HOME is an empty tmpfs in L1).
Spend	Only from the speculation account ; every call reserves from it, under the day ceiling.
Asking the owner	Never. An <code>Approve</code> call is refused with its reason, which becomes a line in the offer ("this needs your OK to run ... , so I stopped there"); a <code>Notify</code> call's notice goes to the speculation ledger only.
Memory	Reads recall as the owner's session would; writes nothing until accepted (no memory pass, no consolidation, no judge training data).
AWS	Refused, except S4's one hand call on its own line.

The catalog stays identical. The speculation is offered exactly the owner's tools and system prompt, so its prefix is a cache read; a refused tool returns a plain result ("not available while working ahead: the owner hasn't asked for this yet; note what you would do"). That is result metadata, not a prompt sentence.

Discard by default. A speculation not used within its lifetime is deleted: its workspace, build tree and transcript go outright (a regenerable tree in a trash still fills the disk). Its ledger rows stay.

Never surprise-apply. Only **Use** writes the owner's tree. An opt-in "use on a matching ask" exists only after precision is proven (D5), and even then it applies only what the owner just asked for, says so first, and Undo is one key.

4.2 The speculation workspace

- **A shared clone, not a worktree.** `git worktree add` shares refs, the stash and the index lock with the owner's repository. A speculation instead makes `git clone --shared --no-checkout` into `<state>/spec/<id>/tree` (objects borrowed read-only, its own refs), checks out the owner's `HEAD` detached, and applies the owner's uncommitted changes as a patch. The owner's `.git` is only read. A directory that is not a repository gets a capped copy (a reflink where the filesystem has it) of the files the prediction names and their package.
- **The base** is recorded: the owner's `HEAD`, a hash of each dirty file, and the session position of the prediction.
- **A build tree of its own** at `<state>/spec/<id>/build`.
- **L1 gets one new knob**, `Spec.rw`: read-write binds, accepted only under `<state>/spec/`. Everything else in the view is unchanged.
- **The daemon's `fs.*`** resolves paths into the clone, so the model sees the same paths it would see in the owner's tree.
- **Caps.** One S2 at a time (S3 at most three); 8 GB a speculation with its build tree; a total cap; no start while the disk under the state directory is below the store's disk-health line.

4.3 Idle only, so FAST is never hurt

A speculation **starts** only when all of these hold, and **pauses** the moment one fails: PSI under the gate's lines (CPU `some avg10` < 20%, IO < 10%), read every second; no owner turn and no owner job in flight; 45 s since the owner's last activity; no build-heavy work of the owner's in progress; free disk above the line; room in the account; the kill switch off.

Pre-emption. Any owner input stops the speculation's model loop at its next await and sends `SIGSTOP` to its L1 jobs; they resume at the next idle, or are discarded if the input made them stale. The owner's turn waits on nothing of a speculation's: no lock, no permit (speculation takes CPU-pool permits only when fewer than half are in use), no store sync. Its jobs and driver run at `SCHED_IDLE`, started from a thread that is itself idle-class so no tokio pool inherits that priority by accident.

No syncs on the owner's path. A speculation's transcript lives in its own directory, not the WAL. Only its ledger rows and its provider calls' rows reach the store (so a real cost is never hidden), batched at background priority. A crash loses the speculation, which is fine.

4.4 Re-verification on Use

1. **Same base** (`HEAD` and every touched file unchanged): apply directly; its tests ran on exactly this tree.
2. **Moved base:** a three-way merge of the ready diff onto the current tree, computed in the speculation workspace. Clean: rerun its recorded test commands in L1 on the merged tree; green applies; red returns a card ("it no longer passes on your current tree: [Rebase it] [Show diff] [Discard]"). Conflict: the card names the files; **Rebase it** runs one short speculation turn to resolve, then the tests, then offers again.
3. **Apply** in one frame with pre-images, so **Undo** restores the exact prior bytes.

4. **Tell the conversation:** one node in the owner's session ("applied the speculation X : its diff stat, its tests, its cost") and the speculation's reply as the answer.

4.5 The ledger and the account

- **Rows:** `spec.predicted`, `scored`, `started`, `paused`, `ready`, `offered`, `used`, `undone`, `discarded` (with why), and `refused` (a call the envelope stopped, with its tool and reason). Each carries its cost and its base.
- **One reservation account:** a day line, a cap a run by rung, 30 model loops and 30 minutes of running time for an S2. At a line the speculation stops and is discarded or offered as partial; it never borrows from a session.
- **A kill switch:** `theseus spec off` (CLI, Discord, cockpit), a durable row read before every step.
- **Visible:** `theseus spec log`, the cockpit's "Ready ahead" card, a weekly digest line: predictions, hit rate, used, dollars spent and wasted, minutes saved.

4.6 Failure modes, each with its guard

Failure	Guard
Stale (the tree moved)	The base fingerprint; a file watch on touched paths marks it stale at once; an 8-hour lifetime; the card shows the age; Use re-verifies
A confused offer	Unsolicited offers only on a return from idle, never mid-conversation or mid-reply; a matching ask needs the embedder's line and the judge's yes; auto-use off until precision $\geq 95\%$ over 50; a discarded topic not offered again for 24 h
Cost runaway	The day line, per-rung caps, loop and wall-clock caps, reservations under the day ceiling, self-lowering rungs, the kill switch
Privacy	Private places only; reads only what the owner's private session there may read; no network, so nothing injected reaches an offer; offers go only to the place that predicted them; no memory writes until accepted
Interference	Idle-only start, instant pause, <code>SCHED_IDLE</code> ; no sync on the owner's path; a clone that only reads the owner's <code>.git</code> ; one S2 at a time; disk caps; the specbench's FAST arm
Escape	L1's namespaces and seccomp; <code>Spec.rw</code> only under <code><state>/spec/</code> ; empty egress; empty grants; planted escapes in the specbench; a security review before S2 ships
Confabulated readiness	The card's test line is the L1 job's exit status, never the model's words; the result is a frozen file

5. The offer, on every surface

```
➤ Ready ahead: fix the failing egress_refused test
I changed the refusal's status from 502 to 403 in egress.rs and updated the test; cargo nextest -p theseus-sandbox: 41 passed.
diff +6 -2 in 2 files · base a1b2c3d (unchanged) · $0.42 · 6 min ago
[Use] [Show diff] [Discard]
```

- **Use** applies (section 4.4) and posts the reply as the answer; **Show diff** opens the diff; **Discard** deletes it. A stopped run says why ("I stopped at the build: it needs your OK to fetch from crates.io"). An S1 result is a lighter card ("🚧 Looked ahead: the gate's last red is a timeout in `tests/tasks.rs`, log attached"). S3 shows the best result with "2 other ways" behind a button.
- **When it may speak.** Theseus does the work without asking and speaks first, but never interrupts. On a matching ask the reply leads with the ready result. Unsolicited: at most one card per return from idle (the first message or press after 10 minutes away), three a day a place, never pinging, never during a streaming reply; overnight results ride in the morning note.
- **Per place:** on in private places (the DM, CLI, TUI, trusted guild channels); off and not switchable in shared places; off per repository until the repository is trusted once.

Surface	Offer	Commands
Discord	The card as a Components V2 message in the DM (a silent speech event); a "Ready ahead" section on the house's board; Show diff opens a thread	buttons; <code>/spec on off list</code>
CLI	The card above the prompt on return	<code>theseus spec list show use discard off on log stats</code>
TUI	A one-line toast ("u use · d diff · x discard"); the diff in the TUI's renderer	the keys; <code>/spec</code>
herdr	A row per ready speculation, below needs-you rows	the row's keys
Cockpit	A "Ready ahead" card: ready, running, discarded today, the account, the hit-rate trend	buttons; the kill switch

6. Measurement: the specbench

Two layers, like the asyncbench's.

Layer 1, the harness, on the stand-in model (\$0). Scripted sessions whose next ask is known: time to a useful answer warm against cold (target under 1 s for a matching ask); pre-emption (the owner's turn starts within 50 ms, the speculation's jobs stop within 100 ms); FAST (every row of the north star's speed table with speculation running at idle against off; none may move outside its noise); the envelope (planted attempts to write outside, reach the network, read a secret, post, start a task or install globally, each refused with no trace outside `<state>/spec/`); and re-verification (base unchanged, moved cleanly, moved with a conflict, passing before and failing after).

Layer 2, the behaviour, on the real model (~\$40 a run). About 30 scenarios replayed from real work (scrubbed), each a session prefix plus the owner's actual next ask, sealed from the predictor; ten of them have an unrelated next ask, to measure confused offers. Arms: off, S0, S0 to S1, S0 to S2, S0 to S3. Metrics: hit rate per source, time to a useful answer (median and p95), dollars per accepted speculation and **wasted dollars**, correctness of what Use applies, and the confused-offer rate.

Live: `theseus spec stats` and the cockpit: 7-day hit rate, used, minutes saved, dollars spent and wasted, pauses, refusals, stale discards.

Benchmarks improve, never max. The predictor's suffix is fixed before the first run; a change that raises the hit rate on the specbench but not the live 7-day rate is a loss. Each run gets its report in `docs/benchmarks/` and a row in the omnibus.

7. Rows

What the stopping point already covers. S0's warm parts are rows of the stopping point (epic theseus-6rui) and count as **covered, not new**: the keep-warm and TTL fix (`cache-fix` , theseus-ezeg), the warm provider connection (`warm-first-token` , theseus-tnky), and recall inside its deadline (`recall-in-deadline` , theseus-zoly). Everything below is a new capability, a child of theseus-upf0 labelled `stop-point-out` , **blocked by the benchmark rerun theseus-lc7a**, so the rerun measures Theseus without it.

Wave	Beads	Row	What it builds	Size	Gate and live check
W1	theseus-upf0.1	<code>spec-ledger</code>	<code>spec.*</code> rows, <code>[spec]</code> config, the account, the kill switch, <code>theseus spec log</code>	2	Unit tests; the kill switch read before every step. Live: <code>spec off</code> halts a running shadow loop
W1	theseus-upf0.2	<code>spec-predict</code>	The deterministic signals; the prediction call; scoring and calibration; shadow only	3	Stand-in tests: a prediction leaves the prefix bytes unchanged (a cache read). Live: a week of shadow, hit rate per source
W1	theseus-upf0.3	<code>specbench-1</code>	Layer 1 and its scenarios; the FAST arm	3	Runs in the gate's bench step, \$0
W2	theseus-upf0.4	<code>spec-s0</code>	Only the new parts: recall pre-run and held for the next turn; file and index warm; a TTL refresh on a strong prediction	2	Layer 1: a matching ask's recall served under its deadline from held hits
W2	theseus-upf0.5	<code>spec-s1</code>	Scouting in L1; the scout note handed to a matching turn; the light card	3	Layer 1's S1 envelope scenarios. Live: a week on
W3	theseus-upf0.6	<code>spec-sandbox</code>	The speculation workspace; <code>Spec.rw</code> under <code><state>/spec/</code> ; the <code>spec</code> ceiling; refusals as results; an identical catalog	4	Every planted escape refused; an independent security review before it joins
W3	theseus-upf0.7	<code>spec-s2</code>	The end-to-end run; caps; pause and resume; the ready result	4	Pre-emption ≤ 50 ms; the FAST arm green. Live: S2 on in the DM for a week
W3	theseus-upf0.8	<code>spec-use</code>	Use, Show diff, Discard; re-verification; apply with pre-images; Undo	3	Layer 1's re-verification scenarios. Live: Use then Undo restores the bytes
W4	theseus-upf0.9	<code>spec-surfaces</code>	The card on Discord, CLI, TUI, herdr and the cockpit; pacing; per-place opt-in	3	Paused-clock tests for the new event. Live: a card in the DM
W4	theseus-upf0.10	<code>specbench-2</code>	Layer 2, its report, the omnibus row	2	~\$40 a run, on the owner's yes for the measurement budget
W5	theseus-upf0.11	<code>spec-s3</code>	Several futures, ranked, in parallel under one cap	2	Layer 2's S3 arm earns its place or stays off
W5	theseus-upf0.12	<code>spec-s4</code>	Speculation on the AWS hands, its own sub-line	3	The hands' runaway brake; a live check in the owner's account
W5	theseus-upf0.13	<code>spec-auto-use</code>	Apply on a matching ask once precision $\geq 95\%$ over 50, if the owner turns it on	1	Layer 2's confused-offer rate

About 35 points over five waves. W1 can start the day the rerun reports, in parallel with the self-improvement switch.

8. The owner's decisions (taken at their defaults, 2026-10-09)

#	Decision	Taken
D1	The budget	\$10 a day for all speculation (5% of the \$200 ceiling); caps \$0.10 an S1, \$1.50 an S2, \$3 an S3; S4 on the hands' line at \$2 a day
D2	Rungs on by default	S0, S1 and S2 in private places once their shadow phase passes; S3 off until S2's hit rate is $\geq 40\%$ over 30 scored; S4 off until the hands are live and the owner says so
D3	Idle rules	Start only under the gate's PSI lines, with no owner turn or job in flight, 45 s since the owner's last activity, no build-heavy work, free disk; pause on any input; one S2 at a time
D4	Per-place opt-in	On in the DM, CLI, TUI and trusted guild channels; off and not switchable in shared places; off per repository until trusted once
D5	Applying	Offer only; never applied without Use. "Use on a matching ask" stays off until precision $\geq 95\%$ over 50, then the owner may switch it on
D6	How often it may offer	One unsolicited card per return from idle, three a day a place, never pinging, never mid-reply; a discarded topic not offered again for 24 h
D7	Lifetime	8 hours , stale on any change to a touched file or <code>HEAD</code> ; deleted outright
D8	Network for builds	None , even the operator's egress list; a speculation that needs a fetch stops and says so

9. Risks and open questions

- **The value is in the hit rate.** If S2's real hit rate is under about 25%, wasted dollars dominate; the shadow phase finds out for the cost of predictions alone.
- **Big sessions make S2 expensive.** At 626K tokens each model loop is a $\sim \$0.16$ cache read, so a 20-loop S2 in that DM would be $\sim \$3$ before output. The answer is the arrangement mode (a fresh small session from quoted pieces); the specbench measures both modes' quality.
- **Disk.** A cold Rust build tree per S2 is large; caps and outright deletion are not optional.
- **The cache claim needs a test.** That a same-model, same-catalog, same-system-prompt fork reads the owner's cache is the economics of the whole thing; `spec-predict`'s first test proves the prefix bytes are identical.
- **Open:** whether useful scout notes should also be written to memory (default no); a "speculate on this" verb ("work ahead on the release notes while I'm out"), which is a task with the speculation envelope (likely, in W4).

Sources outside the repo

The design brief this page was cleaned from, its Discord summary, and the cache measurements behind theseus-ezeg are kept in the working investigations outside this repository. Code citations are to this repository at [d348ac67](#).